# 1. Preparación de tu laptop

SIGELAB - Guía de ejecución con Java 21 en Windows

Esta guía corresponde al ZIP actualizado. Está orientada a Windows de 64 bits con procesador Intel/AMD. Si tu laptop utiliza ARM/Snapdragon, revisa la compatibilidad de SQL Server y de las dependencias faciales antes de seguir esta ruta.

## Qué necesitas

- JDK 21: ejecuta y compila la aplicación Swing. No basta con tener un JRE.
- IntelliJ IDEA: abre el proyecto Maven y permite editar el código.
- Python 3.12 de 64 bits: ejecuta el reconocimiento facial. Instálalo con el lanzador py antes de usar el configurador.
- Microsoft ODBC Driver 18 x64: conecta Python con SQL Server.
- SQL Server: utiliza tu instancia de Windows o el contenedor Docker incluido.
- Docker Desktop: solo es necesario para la ruta de base de datos en Docker.
- SQL Server Management Studio (SSMS): sirve para administrar la base; no es el motor SQL Server.
- Webcam, Internet para las descargas iniciales y espacio libre para dependencias, Docker y el modelo facial.

## Dónde se ejecuta cada parte

Java e IntelliJ se ejecutan en Windows. Python también se ejecuta en Windows para acceder a tu webcam. SQL Server puede ejecutarse dentro de Docker o directamente en Windows. No debes ejecutar el servicio de cámara dentro del contenedor SQL.

Elige una sola base para el proyecto: ruta A (página 3, Docker) o ruta B (página 4, SQL Server nativo). Después ambas rutas continúan en la página 5.

Antes de empezar: conecta la laptop a corriente, prueba la webcam con la aplicación Cámara y luego ciérrala. Instala Python 3.12 y ODBC Driver 18 si todavía faltan. Los enlaces oficiales están en la página 10.

# 2. Instalar Java 21 y extraer el proyecto

## Instalar el JDK

1. En la descarga oficial de Microsoft OpenJDK busca la sección OpenJDK 21 y el instalador Windows x64. También puedes usar otra distribución de JDK 21 que ya tengas instalada.
2. Instala el JDK completo. Si ofrece añadir Java al PATH y definir JAVA_HOME, activa esas opciones. Después cierra y abre PowerShell e IntelliJ.
3. Comprueba desde una terminal nueva:

```
java --version
javac -version
Get-Command java
$env:JAVA_HOME
py -3.12 --version
```

Java y javac deben mostrar 21.x; Python debe mostrar 3.12.x. JAVA_HOME debe apuntar a la carpeta del JDK, no a bin. Si aparece Java 17, revisa las Variables de entorno de Windows y el orden de PATH. No es necesario desinstalar otros JDK: selecciona correctamente el que utiliza este proyecto.

## Preparar la carpeta

Extrae el ZIP, por ejemplo en C:\Proyectos. La carpeta de trabajo será C:\Proyectos\sigelab y deberá contener pom.xml, README.md, src, database, facial-service y scripts. No ejecutes archivos desde dentro del ZIP.

```
Set-Location C:\Proyectos\sigelab
Get-ChildItem
.\mvnw.cmd -v
```

El último comando descarga Maven la primera vez y debe mostrar Java version: 21. El proyecto incluye Maven Wrapper; no necesitas una instalación independiente de Maven.

> El pom.xml establece release=21 y Maven exige un JDK de la familia 21. Que IntelliJ use Java 21 no significa que PowerShell también lo use: comprueba ambos. Si ya tienes una base con datos y rostros, conserva sus archivos de configuración y claves.

# 3. Ruta A: SQL Server en Docker

## Iniciar Docker Desktop

Abre Docker Desktop y espera a que el motor esté listo. Utiliza contenedores Linux. Si el instalador pide WSL 2 o reiniciar Windows, completa esos pasos según sus indicaciones.

```
docker version
docker compose version
```

docker version debe mostrar cliente y servidor. Si solo aparece el cliente con un error de conexión, el motor todavía no está listo.

## Configurar SIGELAB

Desde C:\Proyectos\sigelab, ejecuta:

```
py -3.12 scripts/configurar.py
```

Responde: host localhost; puerto 14330; base SIGELAB; usuario sa. Elige una contraseña propia de al menos 12 caracteres con mayúsculas, minúsculas, números y símbolos. Es la contraseña de SQL Server, no la del usuario del programa.

El asistente crea config/application.properties para Java, facial-service/.env para Python y .env para Docker. También genera el token local y la clave facial. No muestra la contraseña ni sobrescribe configuraciones existentes.

## Crear la base y comprobarla

```
powershell -ExecutionPolicy Bypass -File scripts/inicializar-docker.ps1
docker compose ps
```

Espera a que finalice. El estado esperado del contenedor es healthy. El script aplica las tablas y los datos iniciales. Bypass se limita al proceso que ejecuta este script. Si tu equipo impide ejecutar scripts, aplica los SQL desde SSMS como indica la página siguiente.

En SSMS conecta a localhost,14330 con autenticación SQL Server, usuario sa y tu contraseña. Para el certificado local autofirmado, utiliza Trust server certificate si hace falta. Debes ver SIGELAB y 12 tablas.

> Si 14330 está ocupado, cambia el puerto publicado en compose.yaml y el puerto de Java/Python. El puerto interno del contenedor sigue siendo 1433. Continúa en la página 5; no necesitas crear otra base nativa.

# 4. Ruta B: SQL Server instalado en Windows

## Identificar la instancia

Abre SSMS y conecta a tu servidor. Anota host, puerto TCP y credenciales SQL. Tener solo SSMS instalado no basta. No asumas que SQLEXPRESS usa 1433; una instancia puede utilizar un puerto dinámico.

Si TCP/IP no está habilitado: abre SQL Server Configuration Manager, SQL Server Network Configuration y los protocolos de tu instancia. Habilita TCP/IP. En sus propiedades, IP Addresses > IPAll, comprueba el puerto. Para elegir uno fijo libre, vacía TCP Dynamic Ports y establece TCP Port. Reinicia el servicio de esa instancia para aplicar el cambio.

Necesitas un login SQL habilitado. Si solo conectas con autenticación Windows, configura autenticación mixta y un login SQL en tu instancia local, o utiliza Docker. No presupongas que sa está habilitado. Reinicia la instancia si cambias el modo de autenticación.

## Configurar la aplicación

```
Set-Location C:\Proyectos\sigelab
py -3.12 scripts/configurar.py
```

Introduce localhost, el puerto real, SIGELAB y tus credenciales SQL. El configurador requiere una contraseña de al menos 12 caracteres; usa una cuenta de desarrollo adecuada. No ejecutes docker compose ni inicializar-docker.ps1 en esta ruta.

## Ejecutar los archivos SQL en SSMS

Abre y ejecuta con F5, en este orden:

1. database/01_crear_base.sql: crea SIGELAB si no existe.
2. database/02_esquema.sql: crea las tablas funcionales, solo la primera vez.
3. database/03_datos_demo.sql: agrega catálogos y tres equipos.
4. database/05_facial.sql: crea perfiles y auditoría facial.
5. database/04_comprobar.sql: consulta tablas y equipos.

El login utilizado para instalar debe tener permisos de creación. El usuario de ejecución necesita permisos de lectura y escritura sobre las tablas que utilizan Java y Python. Continúa solo si cada script termina sin errores.

> Si ya aplicaste la primera base de SIGELAB de este chat, omite 02 y ejecuta 03, 05 y 04. Si SIGELAB contiene otro esquema, revisa las diferencias antes de continuar. No borres tu base para resolver un error. Deben aparecer CAM-001, PRO-001 y LAP-001 disponibles.

# 5. Instalar e iniciar el servicio facial

## Verificar las dependencias

Python 3.12 x64 y ODBC Driver 18 x64 deben estar instalados. Aunque Java conecte correctamente, Python utiliza ODBC, que es un controlador distinto del JDBC de Java.

```
py -3.12 --version
```

InsightFace puede necesitar compilar extensiones en Windows. Si la instalación lo solicita, instala Microsoft C++ Build Tools con herramientas de desarrollo de escritorio C++ y Windows SDK. Estas herramientas son para Python; seguirás trabajando el programa Java en IntelliJ.

## Instalar el entorno

Desde la raíz del proyecto:

```
powershell -ExecutionPolicy Bypass -File facial-service/setup-facial.ps1
```

Espera hasta que termine. Se crea facial-service\.venv para tu computadora. No copies un entorno virtual de otro equipo ni del ZIP de referencia. El diagnóstico puede señalar fallos si la base, la cámara o el modelo todavía no están preparados.

## Iniciar y comprobar

```
powershell -ExecutionPolicy Bypass -File facial-service/start-facial.ps1
```

Mantén esta terminal abierta. En el primer arranque puede descargarse buffalo_l. Espera a que el servidor inicie en 127.0.0.1:8765. Desde otra terminal:

```
Set-Location C:\Proyectos\sigelab\facial-service
.\.venv\Scripts\python.exe -m scripts.health_check
```

Debes obtener status=ok, modelLoaded=true y database=ok. El comando envía el token. Abrir /health en un navegador sin ese token devuelve 401 y no es una comprobación equivalente.

> FACIAL_API_TOKEN debe coincidir con facial.token de Java. FACIAL_ENCRYPTION_KEY es otra clave y debe conservarse: cambiarla impide descifrar los perfiles registrados. No compartas estas claves ni los archivos .env.

# 6. Configurar IntelliJ IDEA con JDK 21

## Importar y seleccionar el JDK

1. File > Open: selecciona C:\Proyectos\sigelab\pom.xml y ábrelo como proyecto Maven. No abras únicamente src.
2. File > Project Structure > Project: SDK = JDK 21. Language level = SDK default o 21. Si no aparece, usa Add SDK para seleccionar su carpeta o descargarlo.
3. Settings > Build, Execution, Deployment > Build Tools > Maven > Importing: JDK for importer = Project JDK 21.
4. Maven > Runner: JRE = Project JDK 21. Recarga el proyecto desde la ventana Maven y espera las descargas. Los nombres pueden variar ligeramente según la edición de IntelliJ.

El JDK que ejecuta el propio IDE no sustituye el del proyecto o el runner de Maven. La propiedad release del pom controla la versión objetivo de las clases.

## Compilar y ejecutar

En la terminal de IntelliJ o PowerShell:

```
Set-Location C:\Proyectos\sigelab
.\mvnw.cmd -v
.\mvnw.cmd clean package
```

Espera BUILD SUCCESS. Se ejecutan 15 pruebas Java; la prueba SQL opcional se omite por defecto. El paquete se genera en target/proyecto_sigelab-1.0.0.jar.

Abre src/main/java/proyecto_sigelab/Launcher.java y ejecuta main. En Run > Edit Configurations revisa JRE = JDK 21 y Working directory = C:\Proyectos\sigelab. Si apunta a src o target, no encontrará la configuración local.

## Alternativas desde PowerShell

```
.\mvnw.cmd exec:java
# O ejecuta el JAR distribuido:
java -jar dist/SIGELAB.jar
```

Escoge una opción para el primer ensayo. El JAR incluido está compilado para Java 21; Python y SQL Server deben seguir funcionando. Si modificas el código, recompila y usa el nuevo JAR de target para probar los cambios; dist no se actualiza automáticamente.

# 7. Primer ingreso y prueba del sistema

## Administrador inicial

1. En el login pulsa Crear administrador inicial. Completa nombre, apellido, usuario, correo y contraseña. Solo funciona cuando todavía no hay usuarios.
2. Escribe esas credenciales en el login y pulsa Registrar mi rostro por primera vez. Sigue las indicaciones frente a la cámara: posición frontal, perfiles y giro.
3. Tras terminar, pulsa Ingresar con contraseña y rostro. Se realizará una nueva verificación. Registrar un rostro no abre automáticamente la sesión.

Usa buena iluminación y cierra aplicaciones que ocupen la cámara. El menú debe abrirse únicamente si usuario, contraseña, estado y rostro son válidos.

## Ensayo recomendado

- En Equipos comprueba los tres equipos demo disponibles.
- Crea un préstamo con dos o tres equipos y fecha esperada futura.
- Confirma que cambien a PRESTADO y que otro préstamo del mismo equipo se rechace.
- Devuelve solo algunos: el préstamo debe conservar pendientes y seguir ACTIVO.
- Devuelve otro como DANADO: no debe poder prestarse.
- Envíalo a mantenimiento y cierra como REPARADO: debe volver a DISPONIBLE.
- Consulta el historial y la auditoría.

Las fechas de los formularios utilizan hora de Guatemala y formato AAAA-MM-DDTHH:mm. Elige una fecha futura real; no copies una fecha de ejemplo que ya pasó.

## QR, roles y verificación negativa

En Equipos selecciona una fila y guarda el QR como PNG. En Nuevo préstamo usa Leer QR de imagen. No necesitas un escáner ni la webcam para esa lectura QR.

Crea un ENCARGADO desde Usuarios y comprueba que no pueda administrar usuarios ni consultar auditoría. Registra su rostro antes de intentar entrar con esa cuenta.

Prueba también el rostro de otra persona con las credenciales del usuario registrado: debe negar acceso. Las pruebas automáticas no sustituyen esta comprobación real. Completa los resultados en docs/PRUEBAS.md antes de la presentación.

# 8. Volver a abrirlo y conservar los datos

## Secuencia diaria

1. Inicia la base. Con Docker: abre Docker Desktop y ejecuta desde la raíz:

```
docker compose up -d
docker compose ps
```

Con SQL Server nativo, comprueba que el servicio de tu instancia esté iniciado. No repitas el configurador ni la creación de tablas.

2. En otra terminal inicia el servicio facial:

```
powershell -ExecutionPolicy Bypass -File facial-service/start-facial.ps1
```

3. Con Python listo, ejecuta Launcher desde IntelliJ o:

```
java -jar dist/SIGELAB.jar
```

## Cerrar correctamente

Cierra la ventana Java. En la terminal Python pulsa Ctrl+C. Para detener el contenedor conservando datos:

```
docker compose stop
```

> No uses docker compose down -v para cerrar normalmente: -v elimina el volumen. Cambiar la contraseña en .env tampoco cambia la contraseña de una base que ya fue inicializada.

## Copias de seguridad

Conserva el proyecto, el repositorio Git, una copia de la base SIGELAB y la configuración local con la clave facial. El ZIP de código no incluye tu base ni los rostros que registres. Una copia de la base sin la clave original no basta para recuperar perfiles cifrados.

En SSMS puedes hacer clic derecho en SIGELAB > Tasks > Back Up. Si SQL Server está en Docker, la ruta del backup pertenece al contenedor: copia el archivo .bak al equipo después. No la confundas con una carpeta de Windows.

Al mover el sistema a otra laptop, reinstala Python y sus dependencias; no copies .venv. Transfiere la base y claves de manera privada si necesitas conservar usuarios y rostros.

# 9. Resolver errores y ejecutar pruebas

## Errores frecuentes

- Maven pide Java 21 / invalid target release: revisa .\mvnw.cmd -v, JAVA_HOME y Maven Runner. Estás usando otro JDK.
- UnsupportedClassVersionError: el JAR requiere Java 21; comprueba java --version en esa misma terminal.
- No encuentra configuración: Working directory debe ser la carpeta que contiene pom.xml y config.
- SQL no conecta: revisa motor iniciado, puerto TCP y credenciales. En Docker el puerto publicado es 14330.
- Login SQL rechazado: usa autenticación SQL y la contraseña de la instancia elegida, no la del usuario SIGELAB.
- ODBC Driver 18 no encontrado: instálalo en Windows x64. JDBC y ODBC son distintos.
- HTTP 401 facial: el token de Python y Java deben coincidir.
- Cámara ocupada: cierra Cámara, Teams u otras aplicaciones; revisa permisos de cámara para aplicaciones de escritorio.
- InsightFace no se instala: revisa Python 3.12 x64 y herramientas C++.
- Modelo degraded: revisa descarga del modelo, consola y conexión a la base; repite health_check.
- No descifra perfiles: recupera la clave original o registra de nuevo los perfiles mediante un ADMIN.

## Pruebas automáticas

```
.\mvnw.cmd test
Set-Location facial-service
.\.venv\Scripts\python.exe -m pytest -q
```

Para probar SQL Server real, vuelve a la raíz y usa una base de pruebas configurada:

```
Set-Location ..
$env:SIGELAB_INTEGRATION='1'
.\mvnw.cmd -Dtest=SqlServerIntegrationTest test
Remove-Item Env:SIGELAB_INTEGRATION
```

La prueba crea registros temporales propios y trata de eliminarlos al terminar. Si el proceso se interrumpe forzosamente, pueden quedar registros TEST- para revisar.

> Esta entrega comprueba compilación y empaquetado con JDK 21 y 15 pruebas Java. La prueba SQL está omitida aquí. El servicio facial conserva sus 18 pruebas Python aprobadas sin cámara. Windows, Docker, SQL Server real y webcam deben comprobarse en tu equipo.

# 10. Fuentes y lista final

## Documentación oficial

Microsoft OpenJDK: seleccionar versión 21 / Windows x64

IntelliJ IDEA: configuración de Maven y JDK

Docker Desktop para Windows

SQL Server en contenedores

Python para Windows: seleccionar versión 3.12

Microsoft ODBC Driver 18

SQL Server Management Studio

Microsoft C++ Build Tools

## Lista final de preparación

1. Java, javac y Maven muestran versión 21.
2. IntelliJ usa JDK 21 para proyecto, importer, runner y ejecución.
3. SQL Server responde y SIGELAB tiene sus 12 tablas.
4. Python informa health ok, con modelo y base disponibles.
5. Creaste el ADMIN, registraste su rostro e ingresaste.
6. Otra persona no supera la verificación facial de ese usuario.
7. Probaste préstamo, devolución parcial, mantenimiento y QR.
8. Sabes volver a iniciar el sistema y conservar los datos.

Los instaladores pueden cambiar de aspecto o de versión de parche. Selecciona las familias de versiones indicadas y revisa los requisitos oficiales de tu equipo.

## Archivos para la defensa

README.md resume el arranque. MANUAL_USUARIO.md explica las operaciones. MANUAL_TECNICO.md describe arquitectura y reglas. MODELO_DATOS.md contiene diagrama y diccionario. PRUEBAS.md reúne los casos de aceptación.

Esta guía también se incluye en Markdown para copiar comandos. Describe los pasos del proyecto entregado; no significa que la instalación ya se haya realizado en tu laptop.